v2.0.0 · 2026 EDITION

# VulnHive AI

An AI-powered automated penetration testing engine.
24+ specialised vulnerability hunters working in parallel,
on your laptop, for free — explained for everyone.

**25+**
VULN AGENTS

**25**
WAF FINGERPRINTS

**68**
AUTOMATED TESTS

**100%**
OPEN SOURCE

**Complete Project Report & User Guide** — for technical and non-technical readers.

Author: Shyam Kakkad · Repository: github.com/0xShyam-Sec/VulnHive-AI

Document date: 2026-06-10 · Release tag: `v2.0.0`

# Table of contents

# Part 1 — What VulnHive AI is, in plain English

## 1. What's new in v2.0.0 (six things you can see)

If you have used a previous version of VulnHive AI, this section tells you the six biggest improvements in the v2.0.0 release. Each one is explained in plain English; later sections go deeper.

> New 1 — Findings no longer go missing silently
> In the old version, certain bugs hid inside the engine. For example, when you picked "Full" mode in the dashboard, the engine sometimes ran a simpler "Multi-Agent" scan instead. Or the validator that re-checks findings would quietly delete the ones it disagreed with, even if you wanted to see them. **v2.0.0 fixes nine separate places where findings were being thrown away or skipped.** The result: a typical scan now produces 2–5× more findings, and the dashboard tells you when something didn't work instead of just showing "0".

> New 2 — A new confidence system
> Every finding now gets one of five labels: **Confirmed** **High** **Medium** **Low** **False positive**. Nothing is ever deleted. You decide what to look at by toggling filters in the dashboard. See §7.

> New 3 — One bug, many places (Finding + Instance model)
> If the same kind of bug appears on 50 different web pages, the old version collapsed those into a single line — losing the information about *which* pages were affected. v2.0.0 keeps them all: one **Finding** (the issue), many **Instances** (the affected URLs). See §8.

> New 4 — The Scan Health pill
> The dashboard now shows a small badge — **Healthy** **Partial** **Degraded** — that tells you whether everything in the engine ran successfully. Click it to see exactly which sub-step had a problem and why. No more "the scan finished with 0 findings… or did it crash?" ambiguity. See §9.

> New 5 — Live per-agent progress bars
> During a scan, the dashboard now shows a progress bar for *each* of the 25 agents, plus a heartbeat that tells you the connection is alive. Previously, you stared at a single "40%" bar for four minutes wondering if the scan was stuck.

> New 6 — One-click PDF reports that work on every OS
> The old PDF export only worked on macOS (it depended on Google Chrome). v2.0.0 uses **WeasyPrint**, a pure-Python PDF engine that runs on Linux, Windows, and macOS without any browser. The exact PDF you are reading right now was generated by it.

# 2. What is penetration testing? (for everyone)

**Penetration testing** — usually shortened to **pentesting** — means legally and safely trying to break into a computer system to find its security holes *before* real criminals do.

> 💡 **IN SIMPLE TERMS**
> Think of it like hiring a professional burglar to test your house. They try every door, every window, every lock — not to steal anything, but to give you a report saying "*your back window doesn't lock properly, fix it before a real burglar finds it.*"

## Common web vulnerabilities (in plain English)

| Vulnerability | What it means (simple) | Real-world danger |
|---|---|---|
| SQL Injection | Tricking a website's database into revealing or changing data by typing special characters into input boxes. | Attacker steals all user passwords, credit cards, personal data. |
| Cross-Site Scripting (XSS) | Injecting malicious code into a website that runs in other visitors' browsers. | Attacker steals login sessions; hijacks accounts. |
| Authentication Bypass | Finding a way to get in without a valid password. | Attacker logs in as admin without credentials. |
| IDOR | Changing a number in the URL to see someone else's private data (e.g. `/order/123` → `/order/124`). | You view other users' invoices, messages, files. |
| Command Injection | Tricking the server into running the attacker's commands. | Attacker takes full control of the server. |
| Subdomain Takeover | Claiming an abandoned sub-website a company forgot about (e.g. `old.company.com`). | Attacker hosts phishing pages on the company's own domain. |

> ⚠ Important — Legality
> Penetration testing is **only legal** when you own the system, or have explicit written permission to test it. Testing someone else's website without permission is a crime in most countries. VulnHive AI is intended for **education, authorised testing, bug-bounty programs, and CTF competitions**. See §20.

# 3. What VulnHive AI actually does

You give it one web-address (URL), and it does everything automatically:

1. **Crawls** every page, form, and hidden API endpoint on the website.
2. **Logs in** automatically if you provide credentials (supports five different login styles).
3. **Fingerprints** the technology stack and detects which firewall (WAF) protects the site.

4. **Gathers intelligence** — DNS records, subdomains, open ports, known CVEs from public databases.
5. **Attacks** every discovered point with 25 specialised vulnerability agents, running in parallel.
6. **Bypasses firewalls** by automatically mutating any payload that gets blocked.
7. **Validates** each finding to reduce false alarms — but never silently deletes anything (new in v2.0.0).
8. **Chains** vulnerabilities into multi-step attack paths (e.g. *XSS → steal cookie → log in as admin*).
9. **Reports** everything in HTML, JSON, and PDF — with plain-English explanations and code-level fixes.

> **IN SIMPLE TERMS**
> Imagine hiring a team of 25 expert security specialists — one for each type of attack — who work together 24/7, never get tired, cost nothing, and finish in minutes instead of weeks. That is VulnHive AI.

## 4. The problems v2.0.0 fixed

Before v2.0.0, the engine had a class of bugs the team called **silent finding loss**. The tool would *look* like it was working correctly — but findings were disappearing somewhere between discovery and the report. Below is the inventory of fixes, written in plain English.

### Bugs that caused real findings to disappear

| Problem in v1.x | What it meant for users | Fix in v2.0.0 |
|---|---|---|
| **Dashboard ignored the scan mode** | You picked "Full" in the dropdown, but the engine ran the smaller "Multi-Agent" scan instead. Anything that "Full" would have added (Nmap, Nuclei, Shodan) silently never happened. | The worker now honours every option in the dropdown. |
| **Validator silently deleted "false positives"** | An automated re-check labelled some findings as suspect and just deleted them. If it was wrong, you never even knew. | Nothing is deleted. The validator now *labels* the confidence; you choose what to view (see §7). |
| **Over-aggressive de-duplication** | If the same kind of bug appeared on 50 pages, the engine showed it once — losing the information about *which* 50 pages were affected. | The new Finding + Instance model preserves every affected page (see §8). |
| **Nmap, Nuclei, Shodan only ran in "Full"** | Three of the most useful tools didn't run in the default mode. Lost 10–30 findings per scan. | Nuclei and the Playwright crawler now run in the default "Multi-Agent" mode too. Nmap and Shodan stay in "Full" for speed. |
| **The browser crawler found pages but never reported them** | It discovered authenticated endpoints but only stored them as "places I visited". They never became findings, so the IDOR hunter never saw them. | The crawler now emits *IDOR-candidate* findings for every authenticated endpoint. |

| Problem in v1.x | What it meant for users | Fix in v2.0.0 |
|---|---|---|
| **Eight places that swallowed errors silently** | If one agent crashed or the LLM returned malformed output, the scan continued as if nothing happened. You couldn't tell that something had broken. | Every failure now produces a log entry and a row in the scan-errors table. The dashboard shows it via the Scan Health pill (§9). |

### Bugs that made the tool slow or unreliable

| Problem in v1.x | What it meant for users | Fix in v2.0.0 |
|---|---|---|
| **"Parallel" wasn't actually parallel** | The label said "parallel agents", but each one waited for the previous to finish. A scan that should take 60–90 seconds took 5 minutes. | True async parallelism: a verified test shows 3 producers running in 1.13 s (was ~3 s before). |
| **Fragile LLM output parsing** | If the local AI returned slightly malformed JSON, the engine silently lost that finding. | Switched to a library called *Instructor* with grammar-constrained output and auto-retry. The AI cannot return malformed output any more. |

### Things that made the tool hard to use or see

| Problem in v1.x | What it meant for users | Fix in v2.0.0 |
|---|---|---|
| **Dashboard hid most of each finding** | Each finding card only showed 4 fields. To see the payload, the evidence, or the CWE/CVSS scores you had to export the PDF. | Cards now show CWE, CVSS, confidence dots, the payload preview, and a count of affected pages. Click any card → a detail modal with 5 tabs. |
| **"Stuck at 40%" for 4 minutes** | The single progress bar never moved during the attack phase even when agents were finishing. | One progress bar per agent + a heartbeat. You can see exactly which agent is working on which URL. |
| **PDF export only worked on macOS** | The PDF code hard-coded the path to Google Chrome on macOS. Linux and Windows users couldn't export. | WeasyPrint (pure Python, no browser) is now the default. Playwright/Chromium is a flag-controlled fallback for charts that need JavaScript. |

# Part 2 — How it works, step by step

## 5. The 5-phase scan pipeline

Every scan, whatever its mode, runs through the same five phases:

| Phase | What happens | Plain-English analogy |
|---|---|---|
| **1. Discovery** | Web crawler, deep JavaScript analysis, WAF detection, WHOIS/DNS lookup, subdomain enumeration. | "Walk around the property and make a map. Note every door, window, and skylight." |
| **2. Analysis** | The engine looks at everything discovered and decides which vulnerability to test next, prioritising the most valuable targets first. | "Decide which doors look easiest to pick, and try those first." |
| **3. Attack** | 25 vulnerability agents run in *parallel* (new in v2.0.0 — see §6). If the firewall blocks a payload, the adaptive engine mutates it and tries again. | "Send 25 specialists in at once. If one is blocked, try a different lock-pick." |
| **4. Validation** | Every potential finding is re-tested and labelled with a confidence level. **Nothing is deleted** (new in v2.0.0 — see §7). | "Double-check each report. Mark uncertain ones, but keep them in the file." |
| **5. Report** | The exploit-chain engine combines findings into attack paths. HTML, JSON, and PDF reports are produced. | "Write up everything. Highlight the attacks that chain together into something dangerous." |

## 6. The producer model (new in v2.0.0)

The biggest internal change in v2.0.0 is something called the **producer model**. In plain English:

Before v2.0.0, the engine had three different mechanisms for getting findings. The 25 vulnerability agents worked one way. The discovery modules (passive recon, the browser crawler) worked a second way. Imported tools (Nmap, Nuclei) worked a third way. They didn't share infrastructure, so improving one didn't improve the others, and bugs hid in the seams between them.

In v2.0.0, everything that finds a finding is a **producer**. There is one rule for how a producer behaves, one place where findings get persisted, one place that handles errors, and one place that broadcasts progress to the dashboard. That means:

- Adding a new scanner takes ~25 lines of glue code, not ~250.
- A bug fix in the shared pipeline benefits every scanner at once.
- The dashboard's progress bar, error pill, and finding cards all work with every scanner uniformly.

> **IN SIMPLE TERMS**
> Think of it like a factory floor. Before, every product line had its own conveyor belt that didn't connect to anything else — and quality control was inconsistent. Now everything moves on a single conveyor belt, with one quality station, one packaging station, and one shipping label printer. Improvements anywhere in the line improve everything.

### What counts as a producer in v2.0.0

| Producer | What it produces | Where it gets its data |
|---|---|---|
| Passive Recon | Missing security headers, sensitive files (.env, .git, backups), CORS misconfigurations. | Sends harmless requests to the target. |
| Playwright Crawler | Authenticated endpoints as "IDOR candidates"; leaked secrets from JavaScript files. | Real browser navigation. |
| Nuclei (importer) | Hits against 10,000+ vulnerability templates. | Shells out to the `nuclei` binary. |
| Nmap (importer) | One finding per open port, with service version. | Shells out to the `nmap` binary. |
| Shodan (importer) | Known CVEs affecting the target's IP address. | Free Shodan InternetDB endpoint. |
| WAF Detector | Which firewall protects the target (Cloudflare, AWS WAF, etc.). | Sends probe payloads and watches responses. |
| 25 Vulnerability Agents | One finding per confirmed exploit (SQLi, XSS, IDOR, …). | Sends payloads and checks for proof of exploit. |

## 7. The 5-level confidence system (new in v2.0.0)

In v1.x the engine could mark a finding as either "validated" or "not validated", and would delete the "not validated" ones before the user saw them. Two things were wrong with that:

1. The decision was binary — a single coin flip — even though real-world findings sit on a spectrum from "definitely exploitable" to "probably nothing".
2. Deletion was final. If the validator was wrong, you would never know what was lost.

v2.0.0 replaces this with a 5-level confidence axis. Borrowing from **OWASP ZAP** and **DefectDojo** — two industry-standard vulnerability-management tools — the levels are:

| Label | Meaning | How the engine assigns it |
|---|---|---|
| Confirmed | Proof of exploit. The engine has direct evidence that the bug is real. | A payload triggered a database error message; an XSS canary executed; a file outside the web root was successfully read; etc. |
| High | Multiple strong signals. Probably real, but no single piece of definitive proof. | A heuristic test passed and the URL/parameter context matches a known vulnerable pattern. |
| Medium | One signal. Worth investigating, but might be noise. | A single regex pattern matched the response, or the LLM identified a candidate but the engine couldn't run a deterministic test. |
| Low | Possible, but mostly an opinion. Default-hidden in the dashboard. | LLM-only suggestion with no payload-level proof. |
| False positive | The validator believes this is wrong, but it's *kept* for review. | Default-hidden in the dashboard; one click to reveal. |

The dashboard's default view shows **Confirmed + High + Medium**. A toggle at the top of the page reveals Low and False-positive findings, with a count next to it ("12 hidden by current filters"). You always know what you're not looking at.

## 8. How findings are organised: Finding + Instance (new in v2.0.0)

In v1.x, every finding was one row in a flat table. If the same kind of bug appeared on 50 different web pages, the engine kept the first one and threw away the other 49. Bad for two reasons: you didn't know how widespread the problem was, and you couldn't tell whether the second occurrence was different.

v2.0.0 splits the flat row into two parts (borrowed from **DefectDojo**'s data model):

### Finding

One per logical vulnerability. Holds the title, CWE, CVSS, severity, confidence, remediation guidance, references, and a **count of how many places it appears**.

### Finding Instance

One per affected URL. Holds the URL, method, parameter, payload, request, response excerpt, and which tool found it.

> **IN SIMPLE TERMS — "ONE BUG, MANY DOORS"**
> Imagine your security audit finds that every door in your building uses the same broken lock. The "Finding" is "Broken lock model XYZ". The "Instances" are the 50 doors. The old system would have shown you "Broken lock" once and made you guess which 49 other doors are affected. The new system shows "Broken lock" once, with a row that says "50 affected doors — click to see them all".

### Why the split matters in practice

- **For decision-making**: knowing a CORS misconfiguration affects 1 endpoint vs 200 endpoints completely changes the priority.
- **For reporting**: bug-bounty platforms (HackerOne, Bugcrowd) want the full list of affected URLs, not just a representative one.
- **For developers**: the engineer fixing the issue needs to know *every* place to patch.

## 9. The Scan Health pill (new in v2.0.0)

Sometimes a scan finishes with 0 findings. There are three possible reasons:

1. The target really is well-secured.
2. Something in the engine broke half-way through and you didn't notice.
3. You picked the wrong scan mode for the target (e.g. "API" mode on a server-rendered site).

In v1.x there was no way to tell which one. v2.0.0 introduces the **Scan Health pill** in the top-right of the scan detail page:

| Badge | Meaning | What to do |
|---|---|---|
| Healthy | Zero errors recorded across all 30+ producers. | Trust the result. If 0 findings, the target really is clean (for what we could test). |
| Partial | 1–2 producers crashed or returned no data, but most ran fine. | Click the pill to see which producers failed. Usually safe to trust the findings; some might be missing. |
| Degraded | 3+ producers failed, or a fatal error occurred. | Click the pill to investigate. Likely re-run after fixing the underlying issue (e.g. start Ollama, install Nmap). |

Clicking the pill expands a table with every recorded error: which producer, which phase, what went wrong. The errors also stream *live* via Server-Sent Events while the scan is running — so you see problems as they happen, not after the scan finishes.

# Part 3 — The engine: every vulnerability it hunts

## 10. The 25 vulnerability agents, in plain English

Each agent is a specialist that tests for exactly one type of weakness, thoroughly. Below, each agent's **name**, **plain-English description**, and the **technical labels** (CWE = the industry-standard catalogue number; CVSS = the industry-standard severity score from 0 to 10).

### Injection attacks (the classic web bugs)

| Agent | What it finds (plain English) | CWE | CVSS |
|---|---|---|---|
| **SQL Injection** | Tricks the database into running attacker queries. Uses four techniques: error-based, blind boolean, blind time-based, and UNION-select. | CWE-89 | 9.8 |
| **Cross-Site Scripting (XSS)** | Injects JavaScript that runs in other visitors' browsers. Tests reflected, stored, and DOM-based variants. | CWE-79 | 6.1 |
| **Command Injection** | Tricks the server into running operating-system commands (e.g. `; ls /`). | CWE-78 | 9.8 |
| **Server-Side Template Injection (SSTI)** | Injects code into template engines (Jinja, Twig, Handlebars) so they execute attacker code on the server. | CWE-94 | 9.8 |
| **XML External Entity (XXE)** | Uses malicious XML to read files off the server or make it connect to internal services. | CWE-611 | 8.2 |

## 🔑 Authentication & authorisation

| Agent | What it finds (plain English) | CWE | CVSS |
|---|---|---|---|
| **Authentication Bypass** | Finds ways to log in without credentials — hidden admin panels, default passwords, broken session checks. | CWE-287 | 9.1 |
| **IDOR (Insecure Direct Object Reference)** | Changes IDs in URLs (e.g. `/order/123` → `/order/124`) to see other users' data. | CWE-639 | 7.5 |
| **IDOR Advanced** | Tries advanced techniques: UUID enumeration, base64 ID guessing, parallel session abuse. | CWE-639 | 8.1 |
| **JWT (JSON Web Tokens)** | Tries to forge authentication tokens: `alg=none` attacks, weak HMAC keys, key confusion. | CWE-347 | 7.5 |
| **CSRF (Cross-Site Request Forgery)** | Tricks logged-in users into performing actions they didn't intend (e.g. clicking a link that transfers money). | CWE-352 | 6.5 |
| **Mass Assignment** | Sends extra parameters in a form to set fields you shouldn't be able to set (e.g. `is_admin=true` on a registration form). | CWE-915 | 7.5 |

## Infrastructure & network

| Agent | What it finds (plain English) | CWE | CVSS |
|---|---|---|---|
| **Server-Side Request Forgery (SSRF)** | Makes the server fetch URLs the attacker controls — often pointing at internal addresses to reach things behind the firewall. | CWE-918 | 8.6 |
| **Subdomain Takeover** | Looks for abandoned sub-sites whose DNS still points to a third-party service (Heroku, S3, etc.) that you can re-register. 34 takeover signatures. | CWE-200 | 3.7 |
| **HTTP Request Smuggling** | Confuses load-balancers and back-end servers about where one request ends and the next begins, letting the attacker poison other users' responses. | CWE-444 | 8.6 |
| **Open Redirect** | Finds URL parameters that redirect users to attacker-controlled sites — a common building block in phishing. | CWE-601 | 6.1 |
| **Cache Poisoning** | Tricks the cache (CDN) into serving malicious content to *every* visitor. | CWE-444 | 7.5 |

## 🧪 API, logic & information

| Agent | What it finds (plain English) | CWE | CVSS |
|---|---|---|---|
| **GraphQL** | Introspection attacks, query-depth abuse, alias-based rate-limit bypass, batch query abuse. | CWE-200 | 5.3 |
| **API Versioning** | Looks for old API versions (e.g. `/v1/`, `/v2/` `beta/`) that may still respond and lack the security patches the latest version has. | CWE-200 | 3.1 |
| **Business Logic** | Tests for application-specific logic flaws: price manipulation, coupon stacking, race conditions, workflow skipping. | CWE-840 | 6.5 |
| **Rate Limiting** | Detects endpoints that lack protection against brute-force attacks on logins, password resets, OTP entry. | CWE-799 | 5.3 |
| **WebSocket** | Tests real-time connections for missing origin checks, unauthenticated message channels, message injection. | CWE-1385 | 5.3 |
| **Security Headers** | Checks for missing or mis-configured browser protections: X-Frame-Options, CSP, HSTS, Referrer-Policy, Permissions-Policy. | CWE-693 | 4.3 |
| **Sensitive Data Exposure** | Finds exposed files that shouldn't be web-reachable: `.env`, `.git`, database backups, configuration files. | CWE-200 | 5.3 |
| **File Upload** | Tries to upload web shells, polyglot files, and content-type confusions to gain code execution. | CWE-434 | 8.8 |
| **Path Traversal** | Uses `../../` sequences and encoded variants to read files outside the web root (e.g. `/etc/passwd`). | CWE-22 | 7.5 |

# Part 4 — Using VulnHive AI

## 11. System requirements

| Requirement | Minimum | Recommended |
|---|---|---|
| Operating system | macOS / Linux / Windows | macOS or Linux |
| Python | 3.10+ | 3.14 (the project's target) |
| RAM | 8 GB | 16 GB (for local AI) |
| Disk space | 2 GB (core) | 15 GB (with AI models) |
| Internet | Required for cloud features & threat intel | Broadband |
| Local AI (optional) | None (works without) | Ollama with `qwen3:14b` or `deepseek-r1:14b` |

> ⚠ **PYTHON 3.14 NOTE**
> v2.0.0 upgraded the project from Python 3.9 to 3.14. If you're upgrading from v1.x, you must rebuild your virtual environment: `python3.14 -m venv venv` then `./venv/bin/pip install -r requirements.txt`. The new dependencies (Instructor, Pydantic 2.x) require Python 3.10+.

## 12. Installation guide (step by step)

```
# 1. Clone the project from GitHub
git clone https://github.com/0xShyam-Sec/VulnHive-AI.git
cd VulnHive-AI

# 2. Create a Python 3.14 virtual environment
python3.14 -m venv venv
source venv/bin/activate

# 3. Install Python dependencies
pip install -r requirements.txt

# 4. (macOS only) System libraries for the PDF engine (WeasyPrint)
brew install pango cairo

# 4b. (Linux / Ubuntu) Same step
sudo apt-get install -y libpango-1.0-0 libpangoft2-1.0-0 libharfbuzz0b \
                        libcairo2 libgdk-pixbuf-2.0-0 fonts-liberation

# 5. (Optional) Install a free local AI
ollama pull qwen3:14b           # default; great speed/quality balance
ollama pull deepseek-r1:14b     # used for reasoning-heavy agents

# 6. (Optional) Add cloud-AI keys to .env
echo "GROQ_API_KEY=your-key-here" > .env
```

## 13. Command-line guide

### Basic scan

```
python main.py --target https://example.com
```

The simplest possible scan. Uses the default **Multi-Agent** mode (now includes Nuclei + Playwright in v2.0.0).

### Authenticated scan

```
python main.py --target http://localhost:8080 \
    --auth-type form \
    --login-url http://localhost:8080/login.php \
    --username admin --password password
```

For sites behind a login. Five `--auth-type` values are supported: `form`, `basic`, `bearer`, `cookie`, `none`.

### Full pentest (everything turned on)

```
python main.py --target https://example.com \
    --mode full \
    --exploit-chains --adaptive \
    --nmap full --nuclei cve \
    --report-dir ./reports
```

### Choose your AI backend

```
# Free local AI (default)
python main.py --target https://example.com --llm ollama

# Free cloud AI (faster, ~10× quicker than Ollama)
python main.py --target https://example.com --llm groq
```

### Launch the dashboard

```
# Two terminals are needed.
# Terminal 1 — the web server
python main.py --dashboard --port 5001

# Terminal 2 — the worker that runs scans (macOS forking-safety var is required)
OBJC_DISABLE_INITIALIZE_FORK_SAFETY=YES python -m dashboard.worker
```

Then open `http://localhost:5001` in your browser.

# 14. The dashboard, tour by tour

## 14.1 Starting a scan

From the home page, click **New scan**. Fill in the target URL, optionally a login URL + credentials, pick a mode, and click **Start**. The scan is queued and a worker picks it up within a second.

## 14.2 The live scan page

While a scan runs, you see **three tiers of progress** (new in v2.0.0):

1. **Overall percentage** across all five phases, with the elapsed time.
2. **Current phase** — Discovery, Analysis, Attack, Validation, Report.
3. **Per-agent progress rows** — one bar per producer, showing what URL each agent is currently testing.

A small "● live" indicator pulses every 5 seconds — the *heartbeat*. If it stops, you'll see "Connection stalled" so you know the dashboard is no longer receiving updates.

The **Scan Health pill** sits in the top-right (§9). It updates in real time as errors are recorded.

## 14.3 Finding cards

Each finding is a card showing six things at a glance (new in v2.0.0):

| Element | What it tells you |
|---|---|
| Severity badge | Critical / High / Medium / Low / Info — colour-coded. |
| Confidence dots | ●●●●● Confirmed, ●●●●○ High, ●●●○○ Medium, ●●○○○ Low, ●○○○○ False-positive. |
| CWE + CVSS | Industry-standard catalogue number (e.g. CWE-89) + numeric severity (e.g. 9.8). Click CWE → MITRE definition. |
| URL + parameter | The exact endpoint and form field that's affected. |
| Payload preview | The first 80 characters of the payload that triggered the finding. |
| Instance counter | "3 affected endpoints" when the same Finding has multiple Instances (§8). |

## 14.4 The detail modal — five tabs

Click any card → an HTMX-loaded modal opens with five tabs:

1. **Overview** — severity, confidence, CWE link, CVSS, status, rule ID, occurrence count.
2. **Evidence** — the full payload, request/response excerpts, deterministic-probe markers.
3. **Affected (N)** — the table of every **FindingInstance** with URL, method, parameter, payload, source.
4. **Remediation** — plain-English fix guidance, plus code snippets where available.
5. **References** — OWASP / CWE / CVE links and the original rule that fired.

## 14.5 The filter bar

At the top of the findings list (new in v2.0.0), a row of checkboxes lets you toggle:

- **Severity** — Critical, High, Medium, Low, Info.
- **Confidence** — Confirmed, High, Medium, Low, False-positive.
- **Status** — Active, Verified, Out-of-scope, Risk-accepted.

The list updates instantly (via HTMX) without a full page reload. A count of "12 hidden by current filters" tells you how much is being filtered out.

## 14.6 Stopping a scan

The big red **Stop** button cancels the scan. New in v2.0.0: it now responds in about 1 second instead of waiting for the current phase to finish. The worker checks a Redis flag every 0.5 seconds; when set, every producer notices and exits cleanly.

## 14.7 Exporting a PDF

Click **Export PDF** on the scan detail page. The new cross-platform engine (WeasyPrint) produces a print-quality PDF on Linux, macOS, and Windows alike — no browser dependency. Set `VULNHIVE_PDF_ENGINE=playwright` if you need JS-rendered charts.

# Part 5 — The engineering behind it

## 15. Architecture (the moving parts)

The v2.0.0 architecture follows the pattern used by mature security tools like DefectDojo, Faraday, and ProjectDiscovery Cloud. From top to bottom:

## 16. Technology stack

| Layer | Technology | Purpose |
|---|---|---|
| Backend | Python 3.14 | Core programming language (was 3.9 pre-v2.0.0). |
| | httpx + asyncio | Truly async HTTP client + concurrency. |
| | Pydantic v2 | Type-safe data models (Finding, FindingInstance). |
| | structlog | Structured logging with three sinks (console / file / Redis). |
| | Playwright | Headless browser for crawling JS-heavy single-page apps. |
| | SQLite (WAL mode) | Persistence; reversible migrations. |
| AI / LLM | Ollama + qwen3:14b | Free local AI (default). |
| | Ollama + deepseek-r1:14b | Used for reasoning-heavy agents (business logic, auth bypass, OAuth, race conditions, ATO). |
| | Instructor | Pydantic-validated, retry-on-error wrapper around the LLM. **New in v2.0.0.** |
| | Groq / Gemini (optional) | Free cloud AI alternatives. |
| Security tooling | Nmap 7.99 | Network port + service-version scanning. |
| | Nuclei 3.8 | 10,000+ vulnerability templates. |
| | Shodan InternetDB | Free CVE-by-IP lookups. |
| Frontend / reporting | Flask + RQ + Redis | Web server + job queue + pubsub. |
| | HTMX 2.x | Reactive HTML swaps (no JS framework needed). |
| | Bootstrap 5 + custom CSS | Dark, cyberpunk-styled UI. |
| | WeasyPrint | Cross-platform PDF rendering (replaced Chrome). **New in v2.0.0.** |
| | Server-Sent Events | Live progress, findings, heartbeats, errors. |
| Quality | pytest + pytest-asyncio | 68 automated tests (unit + integration + migration). |
| | ruff | Lint + bare-except ban (S110 / S112). **New in v2.0.0.** |
| | GitHub Actions | CI on every push: ruff + 68-test suite + Linux WeasyPrint check. |

## 17. Project journey — three months of building

| Stage | What was built |
|---|---|
| **1. Foundation** | "XBOW-level" engine architecture; 7-phase implementation plan with 15 tasks; central config + a thread-safe shared brain (ScanState). |
| **2. Engine core** | Decision engine, exploit foundations, chain engine, agent registry, Playwright crawler, passive recon. |
| **3. The agent army** | 24 specialised vulnerability agents (SQLi, XSS, SSRF, IDOR, …); 4 payload libraries; chain verifier; API schema inference. |
| **4. Multi-LLM & agents** | Support for Ollama, Groq, Gemini, Anthropic; agent roster grown to 25 with deepseek-r1 → qwen3 model switch. |
| **5. Deeper discovery** | Deep JS crawler (source maps + webpack chunks); subdomain agent with crt.sh, Wayback Machine, async DNS, 34 takeover signatures. |
| **6. WAF & intel** | 25-WAF fingerprinter; WHOIS + DNS + SPF/DMARC; integration with Shodan, NVD, CISA-KEV. |
| **7. External tools** | Nmap + Nuclei integrated. Multi-LLM support productionised. |
| **8. Web dashboard** | Premium cyberpunk SOC dashboard with live progress, animated ring, real-time terminal, findings tables, recon views. |
| **9. AI chat** | An AI assistant that knows your entire scan — answers in plain English, writes fix code, builds exploit chains on request. |
| **10. v2.0.0 uplift** (2026-06) | **This release.** Six categories of fixes: silent finding loss eliminated, true async parallelism, structured LLM output, the Finding+Instance data model, the 5-tier confidence axis, cross-platform PDF, sub-progress UI, Scan Health pill, and a 68-test suite with CI. |

# Part 6 — Reference

## 18. Real-world use cases

### For students & learners

Learn offensive security hands-on. Practise against intentionally-vulnerable apps like DVWA, OWASP Juice Shop, or testphp.vulnweb.com. Understand how real attacks work in a safe, legal environment.

### For developers

Test your own web application *before* launching. Catch security bugs early when they're cheap to fix. The AI assistant suggests remediation code in your language of choice.

### For small businesses

Run security audits without hiring expensive consultants. Get a professional PDF report you can act on or hand to your dev team.

### For bug-bounty hunters

Quickly map a target's attack surface, discover hidden subdomains and endpoints, and identify vulnerabilities to report for cash rewards on HackerOne, Bugcrowd, and Intigriti.

### For security researchers

Automate the repetitive parts of reconnaissance and scanning. Spend your time on deep manual analysis of the most interesting findings the tool surfaces.

### For CTF competitions

Rapidly enumerate targets and find vulnerabilities during time-pressured Capture-The-Flag security competitions.

## 19. Project statistics (v2.0.0)

| | |
|---|---|
| **100+** PYTHON SOURCE FILES | **32K+** LINES OF CODE |
| **25** VULNERABILITY AGENTS | **25** WAF FINGERPRINTS |

| | |
|---|---|
| **68**<br>AUTOMATED TESTS | **10K+**<br>NUCLEI TEMPLATES |
| **4**<br>AI BACKENDS SUPPORTED | **5**<br>CONFIDENCE LEVELS |
| **3**<br>SCAN MODES (FAST, MULTI-AGENT, FULL, BROWSER, API) | **38**<br>COMMITS IN THE V2.0.0 RELEASE |

## 20. Safety, ethics & legal

⚠ Use responsibly

VulnHive AI is a powerful security tool. With that power comes responsibility:

- Only scan systems you own, or have explicit written permission to test.
- Unauthorised scanning of websites is illegal in most countries and can result in criminal charges.
- Always follow **responsible disclosure** — report vulnerabilities privately to the owner; never exploit them.
- This tool is intended for **education, authorised testing, bug-bounty programs, and CTF competitions** only.

Safe practice targets to learn on: **DVWA**, **OWASP Juice Shop**, **scanme.nmap.org**, **testphp.vulnweb.com** — these are built specifically to be tested legally.

## 21. What's next on the roadmap

| Item | Status |
|---|---|
| ~~PDF report generation (one-click branded PDF)~~ | Done in v2.0.0 |
| ~~Per-agent live progress bars~~ | Done in v2.0.0 |
| ~~True parallel agent execution~~ | Done in v2.0.0 |
| ~~Eliminate silent finding loss~~ | Done in v2.0.0 |
| ~~Cross-platform PDF (Linux/Windows/macOS)~~ | Done in v2.0.0 |
| Bugcrowd VRT taxonomy column on every finding | Planned |
| 7-Question triage gate (PASS / DOWNGRADE / KILL) | Planned |
| URL-shape prioritiser (e.g. `/search?q=` → SQLi first) | Planned |
| Scheduled scans (daily / weekly cron) | Backlog |
| CI/CD integration — scan on every GitHub PR | Backlog |
| Cloud-security scanning (AWS / Azure / GCP) | Backlog |
| Source-code scanning (SAST) | Backlog |
| Slack / Discord notifications for critical findings | Backlog |

# 22. Glossary — every technical term explained

**Agent**

A specialised piece of code that hunts for exactly one type of vulnerability — e.g. the SQL Injection agent, the XSS agent. VulnHive AI ships 25 agents.

**API (Application Programming Interface)**

A way for software to talk to other software. Websites use APIs internally to fetch data; modern apps expose them publicly so other developers can build on top.

**API Versioning**

The practice of numbering APIs (`/v1/`, `/v2/`). A common security mistake: the old version stays online but never gets the security patches the new version has.

**Authentication Bypass**

Any technique that lets an attacker access protected functionality without valid credentials — default passwords, broken session checks, hidden admin paths, forced browsing.

**Bug Bounty**

A reward program where companies pay security researchers for responsibly reporting vulnerabilities. Major platforms: HackerOne, Bugcrowd, Intigriti.

**Business Logic Vulnerability**

A flaw in how the application's rules work, not in its code. Example: a coupon code that can be applied twice and reduces the price below zero.

**Cache Poisoning**

Tricking the cache (CDN or reverse proxy) into storing a malicious response that's then served to every subsequent visitor.

**CDN (Content Delivery Network)**

A geographically-distributed cache that sits in front of a website to serve content faster. Examples: Cloudflare, Akamai, AWS CloudFront.

**Confidence (in VulnHive AI)**

A label assigned to every finding indicating how sure the engine is that the bug is real. Five levels: Confirmed / High / Medium / Low / False-positive. See §7.

**CSRF (Cross-Site Request Forgery)**

An attack that tricks logged-in users into performing actions they didn't intend — by getting them to click a link or load an invisible image that sends a request to a site they're authenticated to.

**CTF (Capture The Flag)**

Security competitions where teams race to solve security puzzles. A safe, legal environment to practise offensive security skills.

**CVE (Common Vulnerabilities and Exposures)**

A unique identifier for a publicly-known security vulnerability — e.g. `CVE-2024-1234`. Maintained by MITRE.

### CVSS (Common Vulnerability Scoring System)

An industry-standard score from 0 to 10 measuring how severe a vulnerability is. 0–3.9 = Low, 4.0–6.9 = Medium, 7.0–8.9 = High, 9.0–10 = Critical.

### CWE (Common Weakness Enumeration)

An industry-standard catalogue of *types* of software weaknesses — e.g. `CWE-89` = "SQL Injection". Maintained by MITRE.

### Dedup (deduplication)

The process of recognising that two findings are the "same" issue and merging them. The new Finding+Instance model (§8) makes this smarter: same issue → one Finding, many Instances.

### DNS (Domain Name System)

The system that translates human-readable addresses (`example.com`) into computer addresses (`93.184.216.34`).

### DVWA (Damn Vulnerable Web Application)

An intentionally-vulnerable PHP application built for legal security practice. Comes pre-loaded with most of the bugs VulnHive AI hunts.

### False Positive

A finding the engine reported but which turns out not to be a real vulnerability. v2.0.0 stops *deleting* false positives and starts *labelling* them, so users can review.

### Finding

In VulnHive AI v2.0.0, a logical vulnerability — one row per distinct issue. Holds title, CWE, CVSS, severity, confidence, remediation. Has many Instances. See §8.

### FindingInstance

A specific occurrence of a Finding on a specific URL. Holds the URL, method, parameter, payload, evidence. Many Instances can share one Finding.

### GraphQL

An alternative to REST APIs where clients ask for exactly the data they want in a single query. Comes with its own vulnerability class — introspection, depth attacks, batch abuse.

### HTMX

A small JavaScript library that lets HTML elements make network requests and swap responses into the page — without a heavy JavaScript framework like React or Vue. The VulnHive dashboard is built on HTMX.

### IDOR (Insecure Direct Object Reference)

A vulnerability where changing an identifier in the URL (e.g. `/order/123` → `/order/124`) lets you access other users' data because the server doesn't check ownership.

### Instance

See *FindingInstance*.

**JWT (JSON Web Token)**

A compact way to represent authentication claims, used by many modern APIs. Has its own family of vulnerabilities — `alg=none`, weak HMAC keys, key confusion.

**LLM (Large Language Model)**

The kind of AI behind tools like ChatGPT. VulnHive AI uses an LLM (qwen3:14b by default) to reason about which vulnerabilities to test and how to interpret responses.

**Mass Assignment**

A vulnerability where a server-side application blindly accepts every field a user sends in a form. Example: posting `is_admin=true` to a "register" endpoint.

**Multi-Agent (scan mode)**

The default scan mode. Runs all 25 vulnerability agents plus passive reconnaissance, the Playwright crawler, the WAF detector, and (new in v2.0.0) Nuclei.

**Nmap**

A network scanner that probes hosts for open ports and identifies the services running on them. Industry-standard since 1997.

**Nuclei**

A template-based vulnerability scanner. Each template is a YAML file describing a known issue and how to detect it. Comes with 10,000+ community-contributed templates.

**OWASP (Open Worldwide Application Security Project)**

A non-profit foundation that produces free security resources. Their *OWASP Top 10* ranks the most common web vulnerabilities each year.

**Path Traversal**

Using `../` sequences (or encoded equivalents) in a file-path parameter to read files outside the intended directory.

**Pentesting (Penetration Testing)**

Legally simulating attacks on a system to find security weaknesses before real attackers do. See §2.

**Phase**

One of the five stages of a scan: Discovery → Analysis → Attack → Validation → Report. See §5.

**Producer**

In v2.0.0, anything that emits findings — agents, importers, crawlers. They all share one interface and one pipeline. See §6.

**Pydantic**

A Python library for type-safe data validation. v2.0.0 uses Pydantic to model Findings and Instances, catching shape errors at construction time.

**Rate Limiting**

A defensive measure that limits how many requests a user can make per second. Missing rate limits allow brute-force attacks on logins, OTPs, password resets.

**Redis**

An in-memory database. VulnHive AI uses it for the worker job queue (with RQ) and the pubsub channels that push live updates to the dashboard.

**Remediation**

The fix for a vulnerability. The detail modal's Remediation tab gives plain-English guidance plus code snippets where available.

**RQ (Redis Queue)**

A simple Python job queue backed by Redis. VulnHive AI uses it so the dashboard can queue scans and the worker can pick them up asynchronously.

**SAST (Static Application Security Testing)**

Scanning source code for security bugs without running the app. The opposite is DAST (Dynamic) — what VulnHive AI does today. Planned for a future release.

**Scan Health pill**

A coloured badge in the dashboard showing whether the scan ran successfully end-to-end. See §9.

**Server-Sent Events (SSE)**

A simple way for the server to push live updates to the browser over a long-lived HTTP connection. Used by the dashboard for progress, findings, heartbeats, errors.

**Severity**

How bad a vulnerability is, on a 5-point scale: Critical, High, Medium, Low, Info. Driven by CVSS.

**Shodan**

A search engine for internet-connected devices. VulnHive AI uses its free *InternetDB* endpoint to look up known CVEs affecting an IP address.

**SPF / DMARC**

DNS records that protect against email spoofing. Weak SPF/DMARC means attackers can send fake emails pretending to be the company.

**SQL Injection**

The classic web vulnerability: tricking the database into executing attacker queries. VulnHive AI tests four families: error-based, boolean-blind, time-blind, UNION-based.

**SSRF (Server-Side Request Forgery)**

Making the server fetch URLs the attacker chooses — often pointing at internal services that aren't reachable from outside the firewall.

**SSTI (Server-Side Template Injection)**

Injecting code into template engines (Jinja, Twig, Handlebars) so they execute attacker code on the server. Usually a 9.8 CVSS — full server compromise.

**Subdomain Takeover**

Re-registering an abandoned third-party service (Heroku, S3 bucket, etc.) whose DNS still points at it. Lets the attacker host phishing pages on the victim's domain.

**structlog**

A Python library for structured logging. v2.0.0 uses it so logs are machine-readable JSON and can be streamed to file, console, and the dashboard simultaneously.

**Validator**

The component that re-tests each candidate finding to assign a confidence label. Replaces the older "drop_false_positives" deletion behaviour.

**VRT (Vulnerability Rating Taxonomy)**

Bugcrowd's standard classification for vulnerabilities, used to determine bug-bounty payouts. Adding a VRT column to every finding is planned for a future release.

**WAF (Web Application Firewall)**

A security guard that sits in front of a website and blocks malicious requests. VulnHive AI identifies which of 25 known WAFs is in front of the target and loads bypass strategies.

**WeasyPrint**

A pure-Python library that turns HTML+CSS into print-quality PDFs without needing a browser. v2.0.0 uses it for cross-platform PDF export. This document was rendered by it.

**XSS (Cross-Site Scripting)**

Injecting JavaScript into a web page that then executes in other visitors' browsers. Comes in three flavours: reflected (one-shot), stored (persistent), DOM-based (client-side).

**XXE (XML External Entity)**

An attack against XML parsers that lets the attacker read local files or make the server connect to internal services.

# 23. Conclusion

VulnHive AI started as an idea: "*What if professional penetration testing could be automated, intelligent, and free?*" Over three months and 32,000 lines of code, that idea became a real, working platform.

v2.0.0 took that working platform and made it **honest**. The previous version did most of its job correctly, but it had a class of bugs — silent finding loss, fake parallelism, hidden errors — that quietly undermined the user's trust. The v2.0.0 release found 11 of those bugs and eliminated them. Just as importantly, it added the infrastructure (Scan Health pill, structured logging, confidence labelling, automated tests) so that *future* bugs of the same kind can't hide.

The result combines the thoroughness of 25 specialised vulnerability scanners, the intelligence of modern AI, the power of industry-standard tools like Nmap and Nuclei, and the convenience of a beautiful, honest web dashboard — all running for free on a personal laptop, with no recurring costs and no data leaving your machine.

> **In one sentence:** VulnHive AI puts a 25-person AI security team in everyone's hands — for free, with no silent failures.

VulnHive AI — Complete Project Report & User Guide (v2.0.0)
Author: Shyam Kakkad · github.com/0xShyam-Sec/VulnHive-AI
For authorised security testing and educational use only.